

01 · 팩트시트 — 프로젝트 요약과 인용 가능한 수치

제안서에 들어가는 모든 숫자는 이 문서에서 가져옵니다. 각 수치의 원출처를 함께 적었습니다.

1. 한 문단 요약

DILAB(딜랩)은 소비자 리뷰와 전문가 리뷰를 벡터 데이터베이스에 통합하고, 검색 증강 생성(RAG)으로 모든 판단에 원문 근거가 인용되는 제품 평가 리포트를 자동 생성하는 제품 리뷰 분석 서비스(데모)입니다. 연구팀은 이 서비스를 6일 만에 MVP로 구축·클라우드 배포하고, 이후 벡터 레이어를 pgvector에서 Oracle AI Database 26ai로 이전하는 과정에서 발견한 방법론적 문제(저장소 교체와 임베딩 위치 효과의 혼재)를 요인 분해 방법으로 정식화하여 국제 학술대회 PyGeek 2026에 논문이 채택(구두발표)되었습니다.

2. 무엇을 만들었나 (시스템 개요)

- 수집·분석 파이프라인: 제품명 입력 → 리뷰 자동 수집 → 청킹·임베딩 → LLM 기반 라벨링(카테고리·감성·구매 여정 동시 판정) → 5축 평점(효능·성분·가격·사용감·안전성, 축별 근거 청크 5개 자동 선별)
- RAG 질의응답(Ask): 질문 → 임베딩 → 벡터 유사도 검색(전문가/일반 리뷰를 분리 top-k 후 전문가 우선 결합하는 하이브리드 정책) → LLM이 근거 인용 답변 생성 (근거 밖 내용 금지 프롬프트)
- 운영 아키텍처(M6): 서버리스 엣지(웹·API) + 서버리스 GPU 워커(무거운 분석) + 관리형 벡터 DB — 상시 서버 0대, 운영비 월 \$0
- Oracle 전환(진행 중 과제): 임베딩 모델을 DB 안에 적재(ONNX)해 임베딩·검색이 단일 SQL로 완결되는 in-database 구조 검증 완료

3. 타임라인

시점	마일스톤
2026-05-26	시장 벤치마킹 리서치 완료 (국내 VoC 분석 SaaS 대상)
2026-05-27	MVP 풀스택 구현 완료 — 화면 6종 + 자동 분석 파이프라인
2026-05-28	클라우드 배포(24/7 데모) + 서버리스 하이브리드 전환 — 로컬 워커 폐지, 운영비 월 \$0
2026-06-23	Oracle AI Database 26ai 전환 R&D 착수 — 벡터 적재·검색 동등성 검증(Top-k 순서 완전일치)
2026-07	3조건 비교 실험 + 추론통계 분석 → 논문 작성
2026-08	PyGeek 2026 논문 채택 (구두발표), 카메라레디 제출
2026-08-20	PyGeek 2026 구두발표 (Oral Presentation #1, 홍익대)
2026-08	앱 RAG 경로 Oracle 판 재작성 — 질의 임베딩+하이브리드 검색을 SQL 1회로 통합

4. 인용 가능한 수치표

4.1 실험 설계

수치	의미	출처
1,347개	실험 코퍼스 리뷰 청크 수 (운영 데이터, 한국어, 화장품 도메인)	논문 §3
43개	평가 질의 수 — 실제 서비스 로그 13 + 평가축·구매단계 커버리지용 저자 작성 30	논문 §3
3조건	A: pgvector+외부 BGE-M3(1024d, HNSW 근사) / B: Oracle+동일 벡터(정확검색) / C: Oracle+in-DB ONNX e5-small(384d, 정확검색)	논문 §3
0/1/2 3단계	LLM-judge 관련성 채점 척도 (무관/부분/직접), temperature 0	논문 §3
215건	관련성 채점 총 건수 (43질의 × 상위 5청크) — 검정 단위는 질의별 평균 43쌍	stats.md §1.1

4.2 핵심 결과

수치	의미	출처
0.971	저장소 교체(A→B) Top-10 자카드 — 43질의 중 37개에서 상위 10개 완전 동일	stats.md §3
0.175	임베딩 위치 이전(B→C) Top-10 자카드 — 거의 다른 목록 (17개 합집합 중 3개만 겹침)	논문 §4
p = 0.523 / 0.003	관련성 대응표본 t검정 (저장소 / 위치) — 저장소는 무변화, 위치는 유의	stats.md §1.2
d_z = -0.098 / 0.473	효과 크기 (저장소=무시 가능 / 위치=중간)	stats.md §1.2
1.270 → 1.014점	위치 이전 시 관련성 평균 하락 (2점 만점, 약 20%)	stats.md §1.1
150 → 110 → 947 ms	평균 검색 지연 (A/B/C) — 저장소 교체는 오히려 단축, in-DB 임베딩은 약 8배 증가 (최저 등급 인스턴스 기준)	stats.md §2
98.4%	HNSW 근사검색 recall — 근사 인덱스가 놓친 양 1.6%뿐	논문 §4
p = 0.052 (TOST ±0.15)	저장소 등가성 검정 — 기준 미충족을 정직하게 보고 ("나빠졌다고 볼 근거 없음"까지만 주장)	논문 §4
10,000회, seed 42	부트스트랩 신뢰구간 — 전체 재현 가능	stats.md 헤더

4.3 시스템·운영

수치	의미
월 \$0	M6 이후 운영 비용 (서버리스 무료 티어 내)
24/7	데모 상시 가동 (상주 서버·개인 PC 의존 0)
6일	벤치마킹 → MVP 구축·배포까지 소요 기간 (05-22 착수 기준)
1024d / 384d	외부 임베딩(BGE-M3) / in-DB 임베딩(multilingual-e5-small ONNX) 차원

수치	의미
SQL 1회	Oracle 전환 후 질의 임베딩+하이브리드 검색 왕복 횟수 (기존: 외부 임베딩 1회 + RPC 2회)

- 원출처 파일: 논문 참고자료/ PDF · 통계 리포트 docs/research/paper-results/stats.md · 원자료 docs/research/paper-results/results_raw.json (재현 코드 scripts/paper_stats.py , seed 고정)

5. 성과 목록 (제안서 "연구팀 역량·선행 성과"용)

- 국제 학술대회 논문 채택·구두발표 — PyGeek 2026 (2026-08-20, 홍익대), *A Factor Decomposition Method for Separating Store Substitution and Embedding Placement Effects in Vector Database Migration*
 - 운영 수준 RAG 서비스 MVP — 수집→분석→평가→질의응답 전 파이프라인 자동화, 24/7 공개 데모
 - 상용 DBMS 벡터 전환 실증 — pgvector→Oracle 26ai 검색 동등성 검증(Top-k 순서 완전일치), in-DB ONNX 임베딩 파이프라인 구축
 - 재현 가능한 평가 체계 — 고정 seed 부트스트랩, LLM-judge(temperature 0), 원자료·코드 공개 가능 상태
 - 비용 효율 아키텍처 — 서버리스 전환으로 상시 서버 0대·운영비 월 \$0 달성
- 공개 링크: 데모 <https://dilab.sean111400.workers.dev> · 연구실 소개 페이지 <https://inc.cau.ac.kr/dilab-consumer-expert-rag-review-system/>

6. 기술 스택 요약

계층	기술
웹·API	Next.js (서버리스 엣지 배포)
임베딩	BGE-M3 1024d (서버리스 AI) → Oracle in-DB ONNX multilingual-e5-small 384d
벡터 저장소	PostgreSQL pgvector (HNSW) → Oracle AI Database 26ai (VECTOR 타입, VECTOR_EMBEDDING / VECTOR_DISTANCE)
분석 워커	서버리스 GPU 컨테이너 (수집·라벨링·5축 평점·토픽)
LLM	DeepSeek 계열 (라벨링·RAG 합성·judge)
실험·통계	Python (paired t / Wilcoxon / bootstrap / TOST)

7. 용어 미니 사전 (비전공 심사위원 대비 — 발표 대본에서 발췌)

- RAG(검색 증강 생성)**: 질문과 관련된 문서를 먼저 검색해, 그 근거를 인용하며 답을 생성하는 방식
- 임베딩**: 글을 의미가 담긴 숫자 목록(벡터)으로 바꾸는 것. 비슷한 뜻이면 비슷한 숫자가 됨
- 벡터 데이터베이스**: 그 숫자 목록을 쌓아 두고 "비슷한 것"을 빨리 찾아 주는 저장소
- 근사 검색(HNSW) vs 정확 검색**: 빠르게 찾는 대신 가끔 놓치는 방식 vs 전부 훑는 방식
- in-DB 임베딩**: 임베딩 계산을 DB 밖 서버가 아니라 **DB 안에서** 수행 — 데이터가 경계 밖으로 나가지 않음
- 브리지(중간) 조건**: 기존 벡터를 그대로 새 저장소에 실은 중간 조건 — 저장소 효과와 임베딩 위치 효과를 한 번에 하나씩 분리

- **p값**: 두 조건이 똑같다고 가정할 때, 관측된 만큼의 차이가 우연으로 얼마나 자주 나오는지의 비율. **크면 판단 오류, 작으면 진짜 차이**
- **효과 크기**: 차이가 "우연인가"(p값)가 아니라 "**얼마나 큰가**"를 표준화한 값 (0.2 작음 / 0.5 중간 / 0.8 큼)
- **등가성 검정(TOST)**: "차이가 없다"를 주장하려면 따로 해야 하는 검정 — 신뢰구간이 미리 정한 띠(± 0.15) 안에 통째로 들어와야 함